



CS & IT ENGINEERING



Algorithms

Divide & Conquer

Lecture No.- 04



By- Aditya sir

Recap of Previous Lecture



Topic

Topic

Binary Search
Merging Algo

Topics to be Covered



Topic

Topic

Topic

Merge Sort

Quick Sort



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW

TTTe



Topic : Merge Sort



(1) Merging Algo:- Summary

Sorted list(L_1) $\rightarrow m$

~~Merge $L_3 \rightarrow (n+m)$~~

Sorted list(L_2) $\rightarrow n$

$[m \leq \text{number of element comparisons} \leq (m + n - 1)]$

$=$

$L_1 \rightarrow L_3 (m+n)$
 $L_2 \rightarrow$

$$m < n$$

$=$

merge Sort $\rightarrow$ Divide + Conquer



$\downarrow$
(Merging Algo)



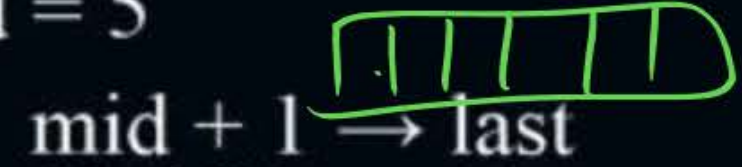
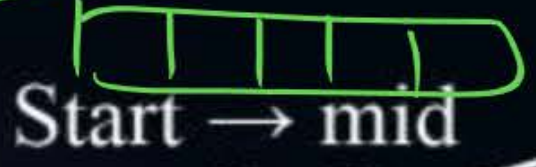
Topic : Merge Sort

Divide and Conquer

A

20	10	50	7	40	80	30	100	15	8
1	2	3	4	5	6	7	8	9	10

P : (1, 10), mid = 5



(1, 5)

(6, 10)

3

8

(1, 3)

(4, 5)

(6, 8)

(9, 10)

2

4

7

9

(1, 2)

(3, 3)

(4, 4)

(5, 5)

(6, 7)

(8, 8)

(9, 9)

(10, 10)

(1, 1)

(2, 2)

50

7

40

(6, 6)

(7, 7)

100

15

8

20

10

80

30

10 20

$$\frac{6+10}{2} = 8$$

$$\frac{6+8}{2}$$

$$m = \left\lfloor \frac{l+h}{2} \right\rfloor$$

$$\frac{1+2}{2}$$

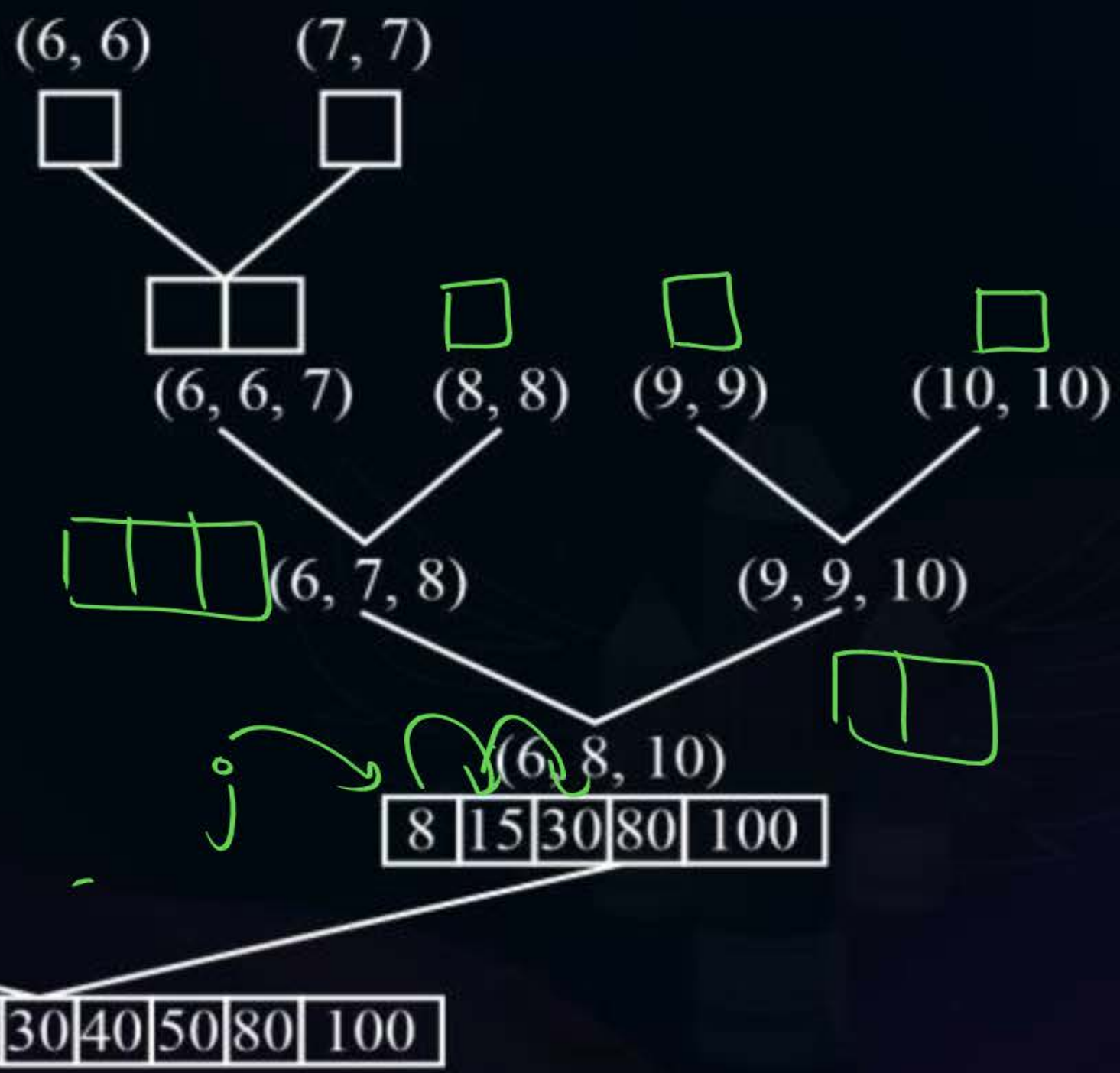
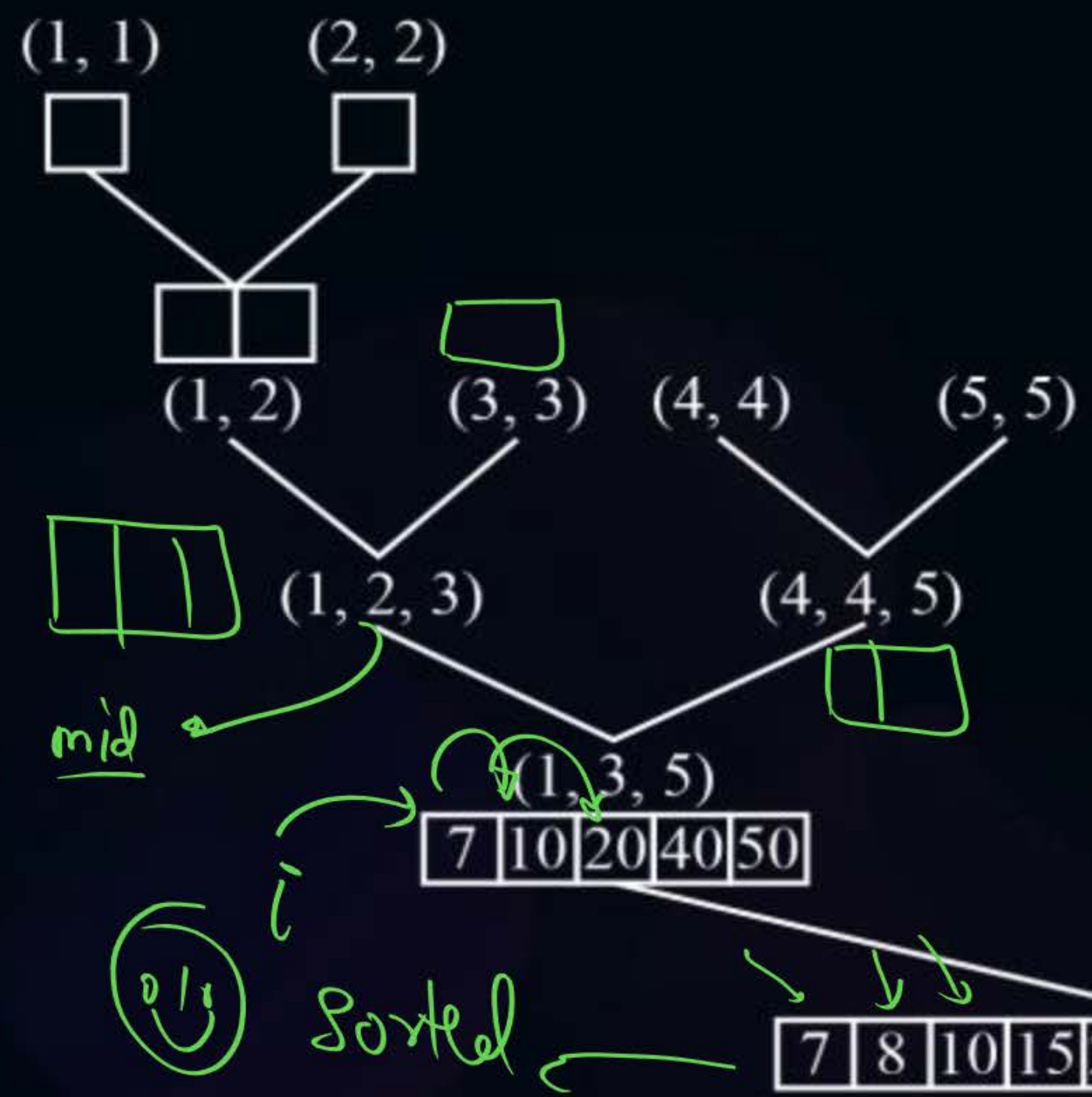
indices

(1, 10)

i/p



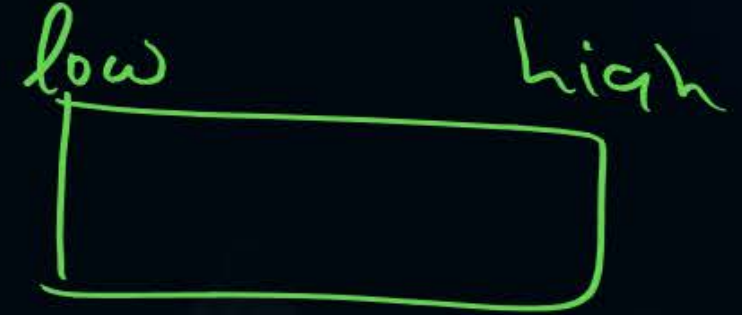
Topic : Sorting Algorithms





Topic : Divide and Conquer

1. Algorithm MergeSort(low, high)
2. //a[low: high] is a global array to be sorted.
3. //Small(P) is true if there is only one element
4. //to sort. In this case the list is already sorted.
5. {
6. → if (low < high) then // If there are more than one element
7. {
8. // Divide P into subproblems.
9. // Find where to split the set.





Topic : Divide and Conquer

10. $mid = \lfloor (low + high) / 2 \rfloor;$

11. // Solve the subproblems.

12. Rec MergeSort(low, mid); $\rightarrow T(n/2)$

13. Rec MergeSort(mid + 1, high); $\rightarrow T(n/2)$

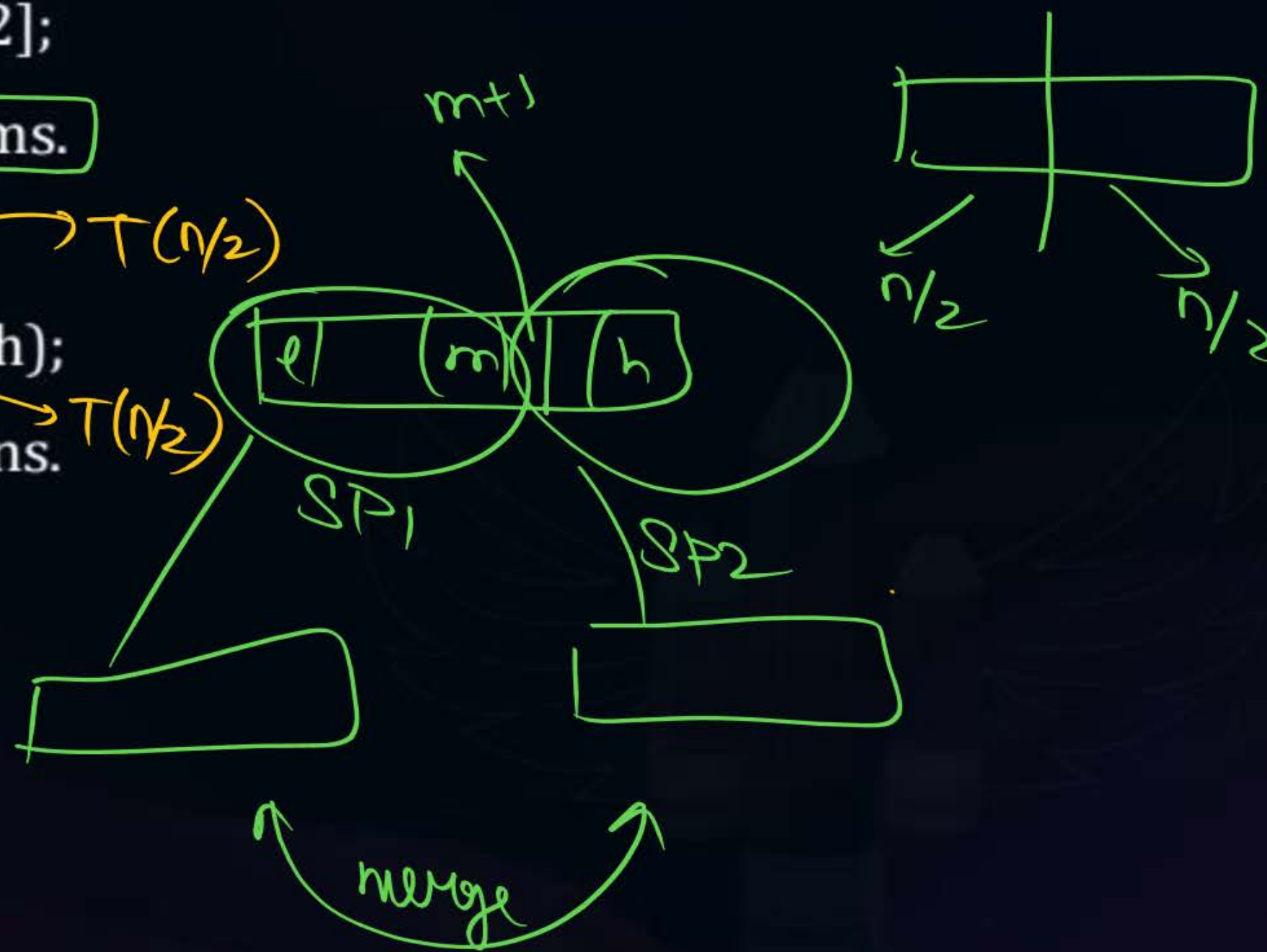
14. // Combine the solutions.

15. Merge(low, mid, high);

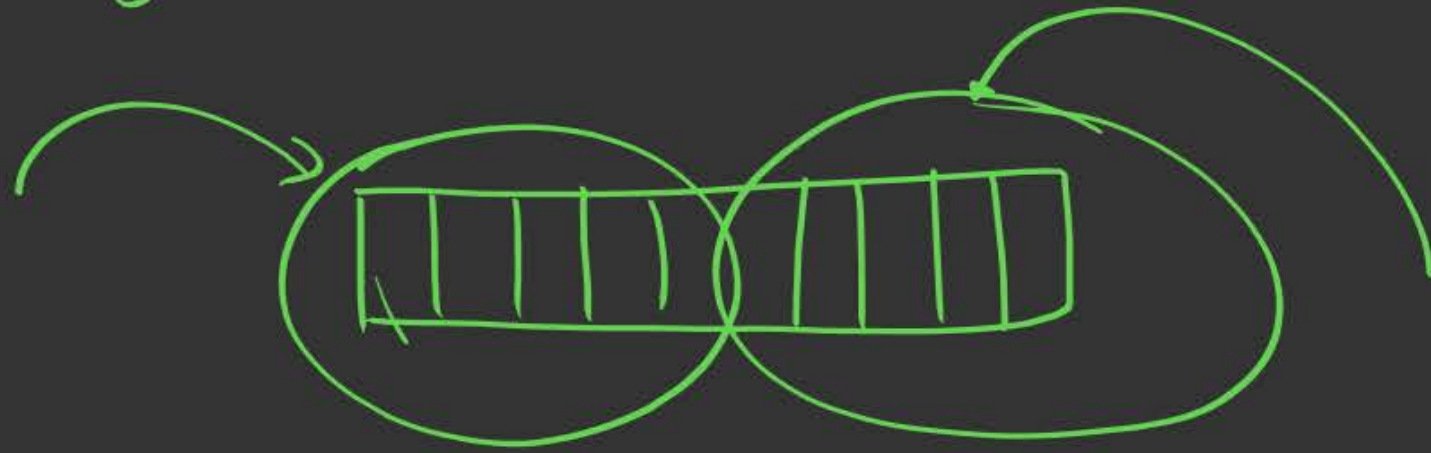
16. }

17. }

$O(n)$



merge Algo



li. $d \rightarrow m$, li. $(m+1) \rightarrow h$

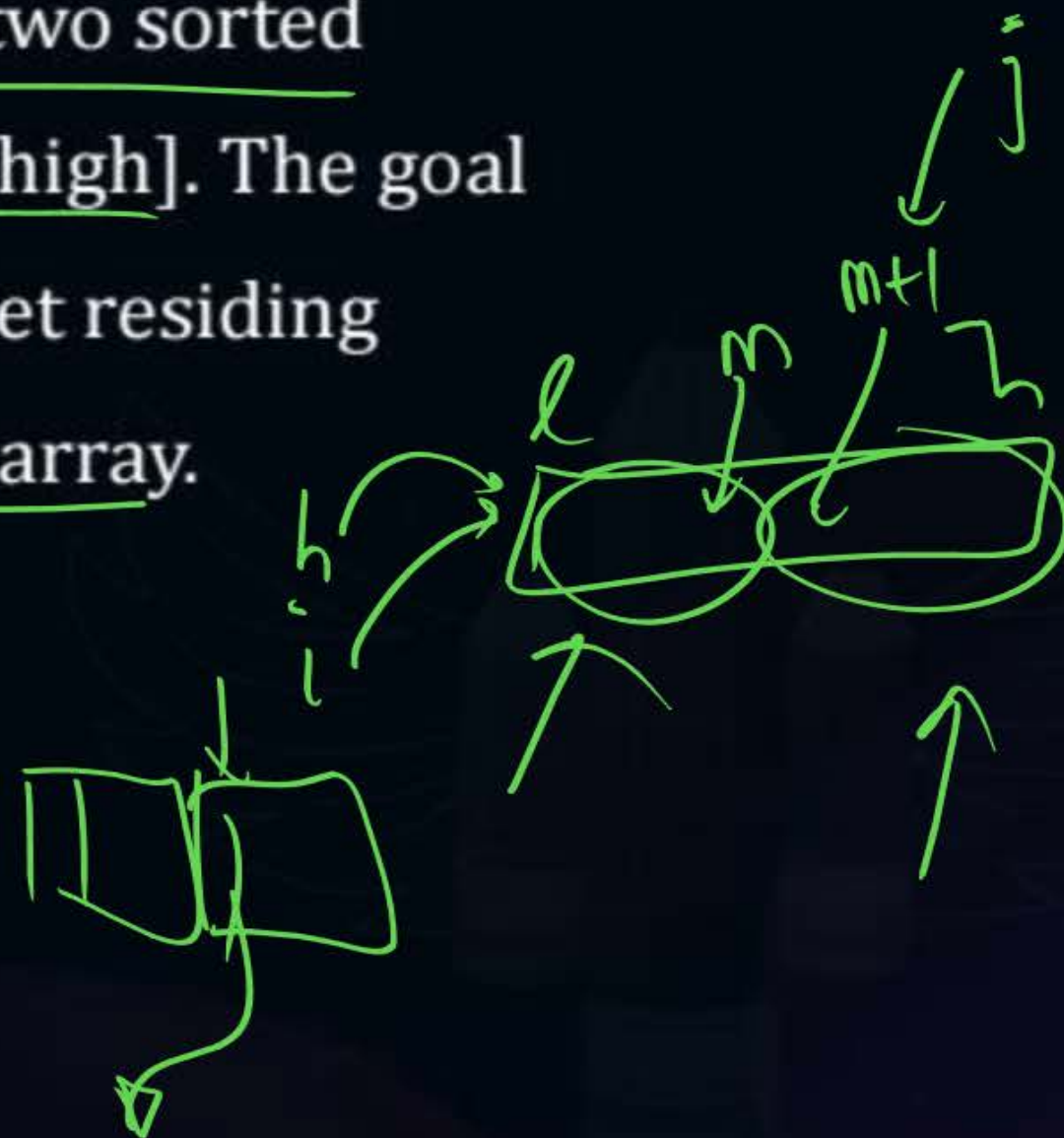
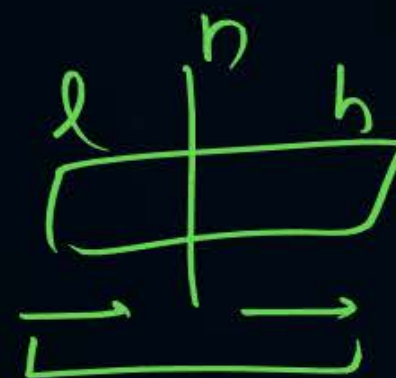
Value of
Return



Topic : Divide and Conquer

1. Algorithm Merge(low, mid, high)
2. //a[low: high] is a global array containing two sorted
3. // subsets in a [low: mid] and in a[mid + 1: high]. The goal
4. // is to merge these two sets into a single set residing
5. // in a[low: high], b[] is an auxiliary global array.
6. {
7. h:= low; i := low; j := mid + 1;
8. while ((h ≤ mid) and (j ≤ high)) do
9. {
10. if (a[h] ≤ [j]) then

→ $O(n)$ → v.i.m.p





Topic : Divide and Conquer



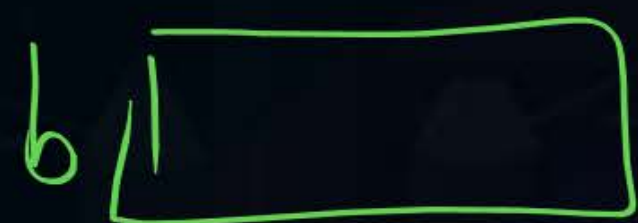
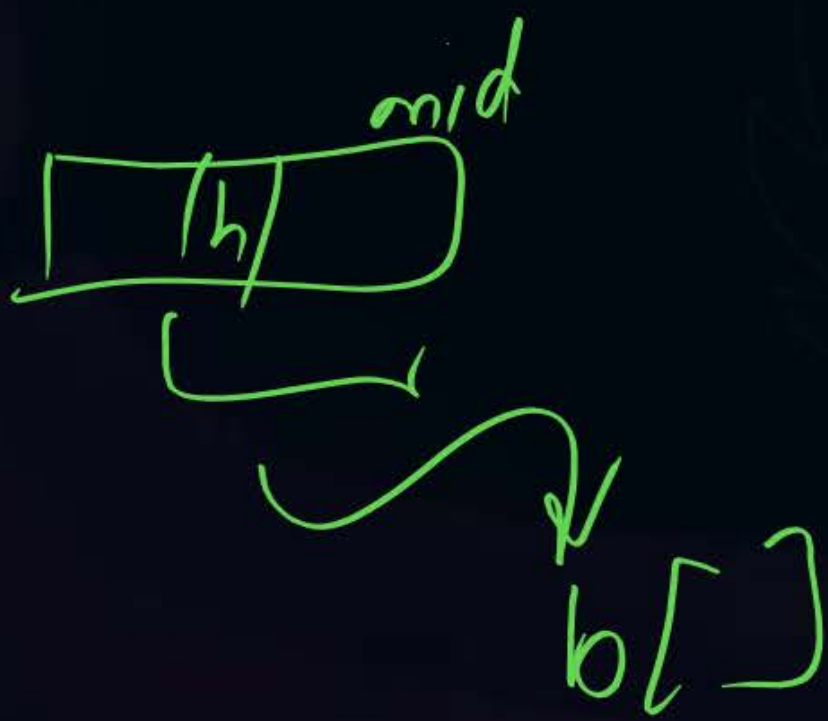
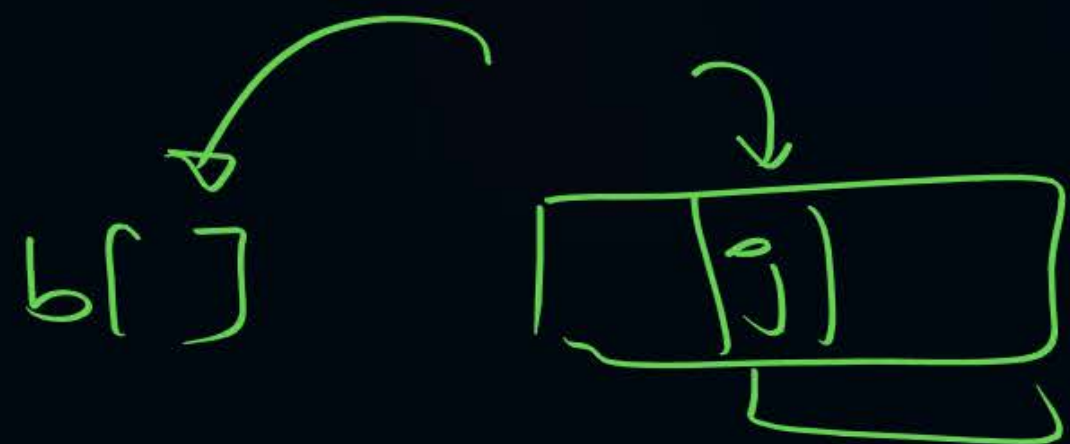
```
11.  {
12.  |  b[i] := a[h]; h := h + 1;
13.  }
14.  else
15.  {
16.  |  b[i] := a[j]; j := j + 1;
17.  }
18.  i := i + 1;
19.  }
20.  if (h > mid) then
```





Topic : Divide and Conquer

```
21. for k := j to high do
22. {
23.     b[i] := a[k]; i := i + 1;
24. }
25. else
26. for k := h to mid do
27. {
28.     b[i] := a[k]; i := i + 1;
29. }
```



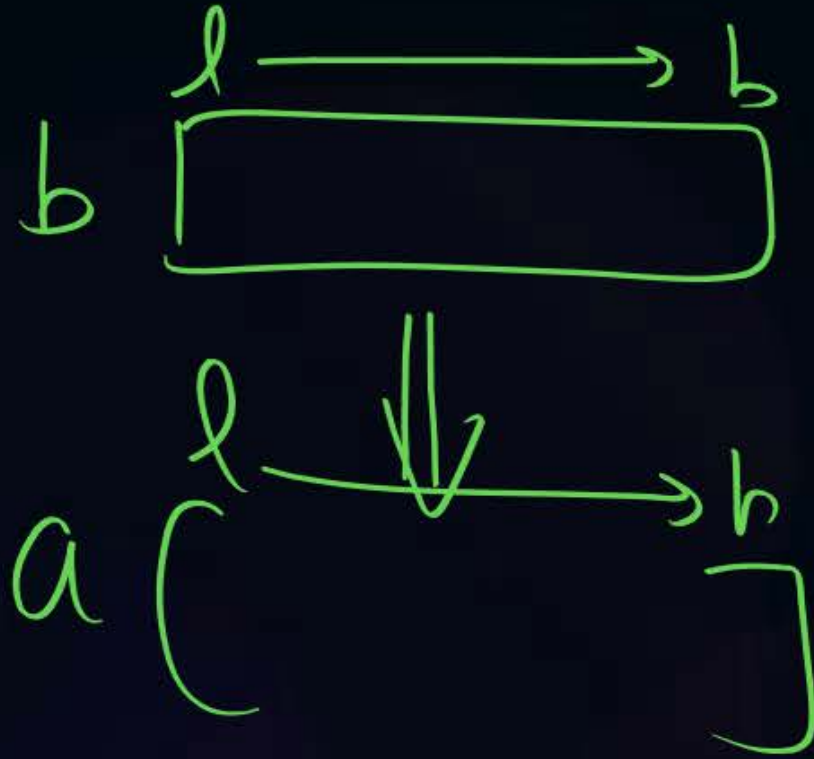


Topic : Divide and Conquer



30. for k := low to high do a[k] := b[k];

→ 31. }



P.Y.Q

#Q. For merging two sorted lists of size m and n into a sorted list of size $m + n$, we require comparisons of

- A** $O(m)$
- B** $O(n)$
- C** $O(m + n)$
- D** $O(\log m + \log n)$

W.C

$$\min = \min(m, n)$$

$$\max = m + n - 1 \Rightarrow O(m + n - 1) = \underline{\underline{O(m+n)}}$$



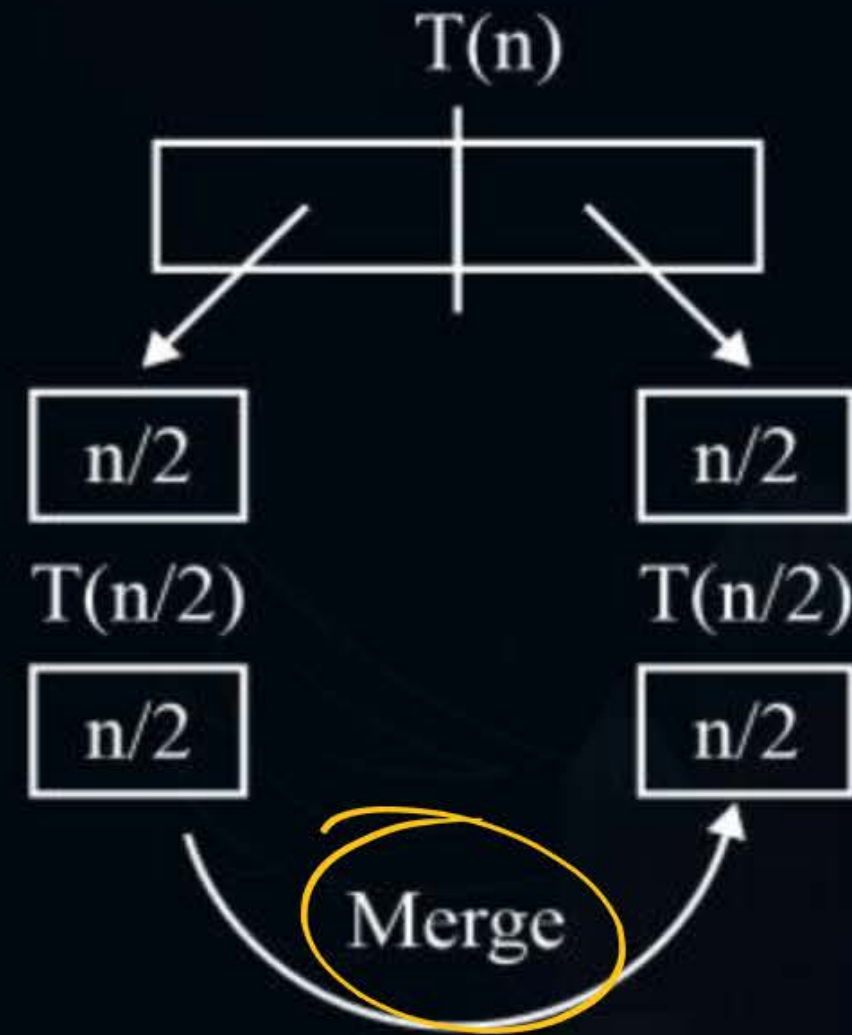
Topic : Performance of Merge Sort

(1) Time Complexity:

Merging :-

Best Case :- $n/2 \Rightarrow O(n)$

Worst case :- $\left(\frac{n}{2} + \frac{n}{2} - 1\right)$
 $= (n - 1) \rightarrow O(n)$





Topic : Performance of Merge Sort

Let $T(n)$ represent time Complexity of Merge Sort on $A[1 : n]$

$$T(n) = \underline{C}, n = \underline{1} \text{ (Small problem)}$$

$$T(n) = 2T(n/2) + \underline{b \times n}, n > 1$$

Solution:-

$$T(n) = 2T(n/2) + b * n, n > 1$$

$$T(n) = 2\left(2T\left(\frac{n}{2^2}\right) + \frac{bn}{2}\right) + b \times n$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + bn + bn$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + 2b \times n$$

$$= 2^3 T\left(\frac{n}{2^3}\right) + 3b \times n$$

Step3 :- $T(n) = O(n \log_2 n)$ every case

General:-

$$T(n) = 2^K T\left(\frac{n}{2^K}\right) + b \times K \times n$$

For B.C,

$$\frac{n}{2^K} = 1$$

$$n = 2^K \rightarrow K = \log_2 n$$

$$T(n) = n \times T(1) + b \times n \times \log_2 n$$

$$= n \times C + \underline{\underline{b \times n \log_2 n}}$$



Topic : Merge Sort

TC

Best Case : $\Omega(n \log_2 n)$

Worst Case : $O(n \log_2 n)$

$O(n \log_2 n)$

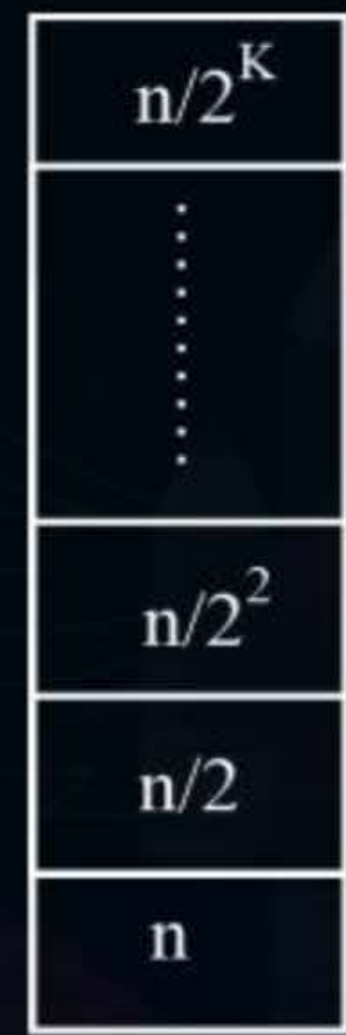
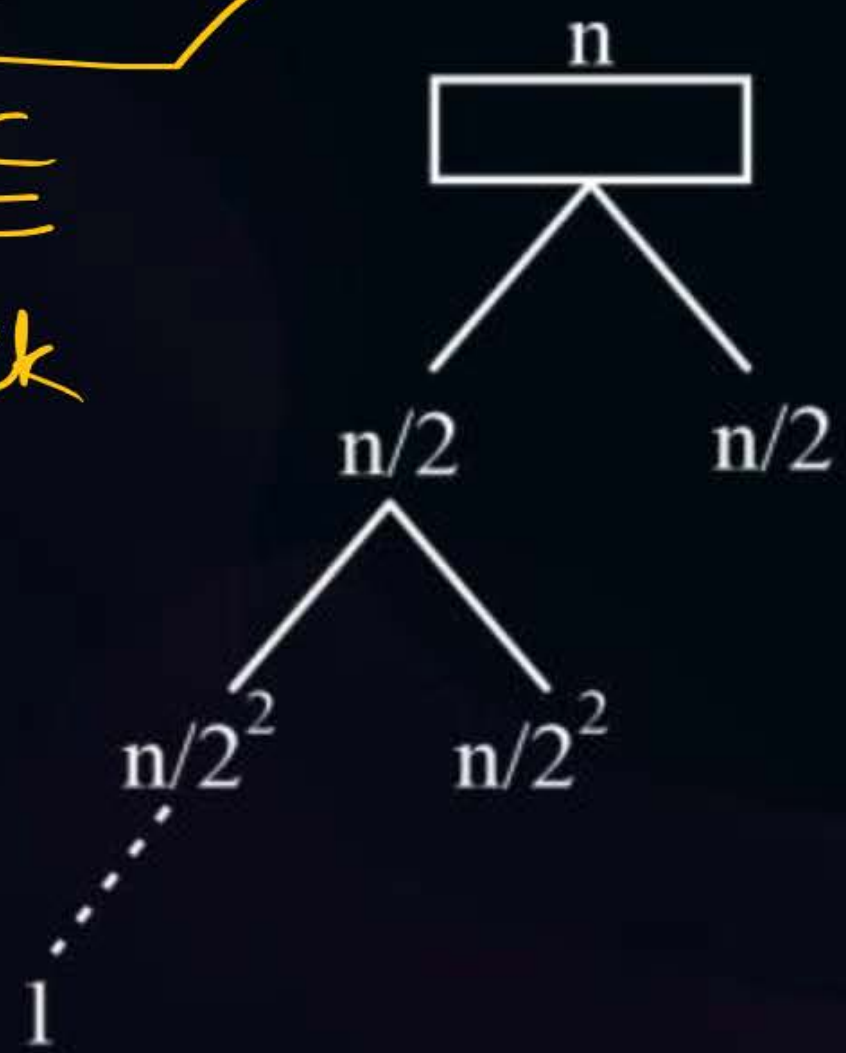
Max height of

Sc

Recursion ~~short~~ Stack
during
~~Print during~~ MS(1, n)

Will be

$\log_2 n$



$$n/2^K = 1$$

$$2^K = n$$

$$K = \log_2 n$$



Topic : Space Complexity Analysis

(1) Temporary/Auxiliary

Array 'B' used for merging

$A = [1 \rightarrow n]$

$B = [1 \rightarrow n]$

$O(n)$

(2) Recursion Stack

$\Rightarrow O(\log_2^n)$

Over all Space Complexity = $O(n + \log_2^n)$
 $= O(n) \Rightarrow$ not inplace

Stable

Performance Analysis of Merge Sort → Summary

Time Complexity $\Rightarrow O(n \log n)$ {Time efficient}

But Space Complexity $\Rightarrow O(n)$ {Space inefficient}

$\Rightarrow$ Merge Sort is stable. (not inplace)

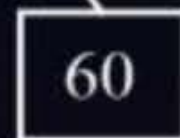
Version-2 :- Bottom-up Merge Sort / 2-way

(Special logic)

A = [31, 25, 16, 60, 34, 40, 86, 24]



Sorted Output



pass 2:

pass 1:-



If equal – pairing not possible

Ex:-

5	20	7	2	8	3
---	----	---	---	---	---

2	3	5	7	8	20
---	---	---	---	---	----

Sorted

$$\underline{\underline{n = 2^k}}$$

pass 20/p:

2	5	7	20
---	---	---	----

5	20
---	----

2	7
---	---

3	8
---	---

5

20

7

2

8

3

pass 10/p:

~~3 8~~



2 mins Summary



Topic

Topic

Topic

Topic

Topic

Merge Sort



THANK - YOU